

mastermind documentation

Version 0.2.1

Contents

classes	1	_theme-builder	11
class	1	utils	12
row	2	_try-upgrade	12
column	3	parse	12
group	3	java	12
draw	3	uml-member	12
_register-diamond-filled	3	uml-class	13
_draw-class	4	uml-relation	14
_draw-relationship	4	uml-package	15
_from-side-to-side	4	uml-diagram	15
draw-uml-diagram	5	class-types	16
draw-source-uml-diagram	5	relation-types	16
lib	6	visibilities	16
csharp	6	relationships	16
_relationship	6	parse-visibility-keyword	16
inheritance	7	is-primitive-type	17
dependency	8	extract-between	17
association	8	parse-stereotype	17
aggregation	9	parse-generics	17
composition	10	parse-relation-side	17
implementation	10	_get-visibility	17
ir	11	_sourcecraft-to-mastermind	18
theme	11	themes	18
		_try-upgrade	18
		parse	18

classes

class

Class object.

Parameters

```
class(  
  name: string,  
  center: array,  
  type: string none,  
  attributes: array,  
  methods: array  
) -> dictionary
```

name string

Name of the class.

center `array`

Center of the class.

Default: `(0, 0)`

type `string` or `none`

Type of the class.

Default: `none`

attributes `array`

Attributes of the class.

Default: `()`

methods `array`

Methods of the class.

Default: `()`

row

Utility to draw the given classes in a row.

Parameters

```
row(  
  center: array ,  
  gap: length ,  
  ..classes: class  
) -> array
```

center `array`

Center of the row of classes.

gap `length`

Gap between classes.

..classes `class`

The classes to draw.

column

Utility to draw the given classes in a column.

Parameters

```
column(  
  center: array,  
  gap: length,  
  ..classes: class  
) -> array
```

center array

Center of the row of classes.

gap length

Gap between classes.

..classes class

The classes to draw.

group

Utility to group the given classes.

Parameters

```
group(  
  center: array,  
  ..classes: class  
) -> array
```

center array

Center of the group of classes.

..classes class

The classes to draw.

draw

_register-diamond-filled

Custom diamond filled mark.

Parameters

```
_register-diamond-filled() -> content
```

`_draw-class`

Draw a class.

Parameters

```
_draw-class(  
  class: class,  
  theme: theme  
) -> content
```

class class

The class to draw.

theme theme

The theme of this diagram.

`_draw-relationship`

Draw a relationship.

Parameters

```
_draw-relationship(  
  relationship: relationship,  
  theme: theme  
) -> content
```

relationship relationship

The relationship to draw.

theme theme

The theme of this diagram.

`_from-side-to-side`

Draw a line from a class side to another.

Parameters

```
_from-side-to-side(  
  from-rect: string,  
  to-rect: string  
) -> array
```

from-rect `string`

Starting class.

to-rect `string`

Ending class.

draw-uml-diagram

Draw a UML diagram.

Parameters

```
draw-uml-diagram(  
  classes: class ,  
  relationships: relationship ,  
  theme: theme  
) -> content
```

classes `class`

The classes to draw.

relationships `relationship`

The relationships to draw.

theme `theme`

The theme of this diagram.

Default: `themes.blueprint`

draw-source-uml-diagram

Draw a UML diagram by parsing a source code file.

Parameters

```
draw-source-uml-diagram(  
  sources: array ,  
  theme: theme ,  
  parser: "java" "csharp"  
) -> content
```

sources `array`

The source files.

theme `theme`

The theme of this diagram.

parser `"java"` or `"csharp"`

The parser to apply.

lib

csharp

_relationship

Basic relationship.

Parameters

```
_relationship(  
  from-rect: string ,  
  to-rect: string ,  
  line: array ,  
  start-mark: string none ,  
  end-mark: string none ,  
  from-side: "north" "east" "south" "west" ,  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect `string`

Starting class.

to-rect `string`

End class.

line `array`

Line.

Default: `()`

start-mark `string` or `none`

Starting line mark.

Default: `none`

end-mark `string` or `none`

Ending line mark.

Default: `none`

from-side `"north"` or `"east"` or `"south"` or `"west"`

Starting side.

Default: `none`

to-side `"north"` or `"east"` or `"south"` or `"west"`

Ending side.

Default: `none`

inheritance

Create an inheritance relationship.

Parameters

```
inheritance(  
  from-rect: string ,  
  to-rect: string ,  
  from-side: "north" "east" "south" "west" ,  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect `string`

Starting class.

to-rect `string`

Ending class.

from-side `"north"` or `"east"` or `"south"` or `"west"`

Starting side.

Default: `none`

to-side `"north"` or `"east"` or `"south"` or `"west"`

Ending side.

Default: `none`

dependency

Create an inheritance relationship.

Parameters

```
dependency(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: none

to-side "north" or "east" or "south" or "west"

Ending side.

Default: none

association

Create an inheritance relationship.

Parameters

```
association(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect `string`

Ending class.

from-side `"north" or "east" or "south" or "west"`

Starting side.

Default: `none`

to-side `"north" or "east" or "south" or "west"`

Ending side.

Default: `none`

aggregation

Create an inheritance relationship.

Parameters

```
aggregation(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect `string`

Starting class.

to-rect `string`

Ending class.

from-side `"north" or "east" or "south" or "west"`

Starting side.

Default: `none`

to-side `"north" or "east" or "south" or "west"`

Ending side.

Default: `none`

composition

Create an inheritance relationship.

Parameters

```
composition(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: none

to-side "north" or "east" or "south" or "west"

Ending side.

Default: none

implementation

Create an implementation relationship.

Parameters

```
implementation(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect `string`

Ending class.

from-side `"north" or "east" or "south" or "west"`

Starting side.

Default: `none`

to-side `"north" or "east" or "south" or "west"`

Ending side.

Default: `none`

ir

theme

Theme object.

Parameters

```
theme(  
  inset,  
  body-fill,  
  header-text-fill,  
  radius,  
  stroke,  
  header-fill,  
  line-color,  
  visibility-fill,  
  line-thickness,  
  mark-size,  
  body-text-fill,  
  line  
) -> theme
```

theme-builder

Create a theme

Parameters

```
theme-builder(  
  inset,  
  radius,  
  thickness,  
  mark-size,  
  primary-color,  
  secondary-color,  
  tertiary-color  
) -> theme
```

utils

_try-upgrade

Try to upgrade the relation type for target in field-rels. Only upgrades if new-type has strictly higher priority. Returns the (possibly updated) field-rels dict.

Parameters

```
_try-upgrade(  
    field-rels,  
    target,  
    new-type  
)
```

parse

Parse a C# source code file.

Parameters

```
parse(source) -> dictionary
```

java

uml-member

Creates a UML member (attribute or method).

Parameters

```
uml-member(  
    name: string,  
    return-type: string none,  
    visibility: string,  
    modifiers: array,  
    kind: string,  
    params: string none  
) -> dictionary
```

name string

Member name (e.g., "getNome")

Default: ""

return-type string or none

Type for fields, return type for methods

Default: none

visibility string

"public", "private", "protected", "package"

Default: "package"

modifiers `array`

e.g., (“static”), (“abstract”),

Default: ()

kind `string`

“field” or “method”

Default: `"field"`

params `string` or `none`

Parameter text for methods (e.g., “String nome, int idade”)

Default: `none`

uml-class

Creates a UML class / interface / enum / abstract class.

Parameters

```
uml-class(  
  name: string ,  
  type: string ,  
  stereotype: string none ,  
  members: array ,  
  generics: string none ,  
  level,  
  order  
) -> dictionary
```

name `string`

Class name

Default: `" "`

type `string`

“class”, “abstract”, “interface”, “enum”, “annotation”

Default: `"class"`

stereotype `string` or `none`

e.g., “Singleton”

Default: `none`

members `array`

Array of uml-member dictionaries

Default: `()`

generics `string` or `none`

e.g., "T extends Comparable"

Default: `none`

uml-relation

Creates a UML relation between two classes.

Parameters

```
uml-relation(  
  from: string,  
  to: string,  
  type: string,  
  label: string none,  
  from-card: string none,  
  to-card: string none  
) -> dictionary
```

from `string`

Source class name.

Default: `" "`

to `string`

Target class name.

Default: `" "`

type `string`

Relation type. See relation-types.

Default: `"association"`

label `string` or `none`

Label on the relation.

Default: `none`

from-card `string` or `none`

Cardinality at source (e.g., “1”).

Default: `none`

to-card `string` or `none`

Cardinality at target (e.g., “*”).

Default: `none`

uml-package

Creates a UML package grouping.

Parameters

```
uml-package(  
  name: string ,  
  classes: array ,  
  style: string none  
) -> dictionary
```

name `string`

Package name

Default: `""`

classes `array`

Array of class names inside this package.

Default: `()`

style `string` or `none`

Visual style hint

Default: `none`

uml-diagram

Root IR node — represents a complete class diagram.

Parameters

```
uml-diagram(  
  classes: array,  
  relations: array,  
  packages: array  
) -> dictionary
```

classes array

Array of uml-class dictionaries.

Default: ()

relations array

Array of uml-relation dictionaries.

Default: ()

packages array

Array of uml-package dictionaries.

Default: ()

class-types array

Valid class types.

relation-types array

Valid relation types.

visibilities array

Valid visibility values.

relationships

parse-visibility-keyword

Java/C# visibility keyword → IR visibility value.

Parameters

```
parse-visibility-keyword(word) -> content
```

is-primitive-type

Check if a type name is a Java/C# primitive (won't generate a relation).

Parameters

```
is-primitive-type(type-name) -> bool
```

extract-between

Extract content between matching delimiters (e.g., "<" and ">").

Parameters

```
extract-between(  
  text,  
  open,  
  close  
) -> string none
```

parse-stereotype

Extract stereotype text from <<Name>>.

Parameters

```
parse-stereotype(text) -> string none
```

parse-generics

Extract generic type from <T> or <T extends Foo>.

Parameters

```
parse-generics(text) -> string none
```

parse-relation-side

Parse cardinality and class name from a relation side. Input: text like Class01, Class01 "1", "many" Class02

Parameters

```
parse-relation-side(text) -> dictionary
```

_get-visibility

Get the visibility of the class

Parameters

```
_get-visibility(e) -> string
```

`_sourcecraft-to-mastermind`

Map sourcecraft types to mastermind's

Parameters

```
_sourcecraft-to-mastermind(  
    classes,  
    relations  
) -> content
```

themes

`_try-upgrade`

Try to upgrade the relation type for target in field-rels. Only upgrades if new-type has strictly higher priority. Returns the (possibly updated) field-rels dict.

Parameters

```
_try-upgrade(  
    field-rels,  
    target,  
    new-type  
)
```

parse

Parse a Java source code file.

Parameters

```
parse(source) -> dictionary
```